

MẠNG NEURON NHÂN TẠO

THUẬT TOÁN HUẤN LUYỆN & TỐI ƯU HÓA

Giải mã toàn diện Momentum, Huber Loss, Adam Optimizer, Focal Loss, Cosine Annealing và Lan truyền ngược Backpropagation qua trực giác đời thực

Tài liệu: Chuyên đề Thuật Toán Huấn Luyện AI | Quy chuẩn: 100% Zero-Emoji & Native Math | Khổ in: A4 7 Trang Chuẩn In Ấn

TRIẾT LÝ SƯ PHẠM | Hiểu Tận Gốc Bản Chất Bằng Ngôn Ngữ Đời Thường

Huấn luyện mạng nơ-ron thực chất giống như **dạy một đứa trẻ tập đi và thích nghi với địa hình hiểm trở**. Các công thức toán học cao siêu chỉ là những quy tắc vật lý rất tự nhiên: quán tính lăn dốc giúp xe không bị kẹt (Momentum), người quan tòa công tâm không nghe tin đồn thất thiệt (Huber), vận động viên biết tăng giảm nhịp thở thông minh (Adam), người thầy dành 90% thời gian kèm học sinh yếu (Focal Loss), và chiếc máy bay hạ cánh mượt mà xuống đường băng (Cosine Annealing).

MỤC LỤC 6 DẠNG BÀI TOÁN TỐI ƯU TRỌNG TÂM

6 Trụ Cột Huấn Luyện AI

Câu	Chủ Đề Trọng Tâm	Ẩn Dụ Thực Tế & Bản Chất Trực Giác	Nghiệm Số & Kết Quả Then Chốt
Câu 1	Gradient Descent with Momentum	Cỗ xe trượt tuyết có quán tính; tích lũy tốc độ dọc hẻm núi, triệt tiêu dao động lắc lư zíc-zắc hai bên sườn đồi, lướt qua cực tiểu địa phương.	$vt = [0.59, -1.47]$; $\theta_t = [0.9941, -0.9853]$; ổn định và tăng tốc độ hội tụ.
Câu 2	Hàm Mất Mát Huber Loss	Vị quan tòa khoan dung; phạt bình phương với lỗi nhỏ nhưng chuyển sang phạt đường thẳng với lỗi lớn, không bị điểm ngoại lai (outliers) bắt cóc.	Huber = $[0.045, 1.000, 1.600]$; MSE trung bình = 2.25; điểm e3=2.1: Huber (1.6) thấp hơn MSE (4.41).
Câu 3	Thuật Toán Tối Ưu Adam	Vận động viên thông thái; kết hợp quán tính hướng đi (Moment 1) và tự điều chỉnh độ dài bước chân theo độ rung lắc (Moment 2); phao cứu sinh epsilon.	$m_1 = [0.1, -0.4]$; $\hat{v}_1 = [0.01, 0.16]$; $\theta_1 = [-0.001, +0.001]$; $\epsilon = 10^{-8}$ chống nổ chia 0.
Câu 4	Focal Loss & Hard Example Mining	Người thầy tập trung kèm học sinh yếu; dim mất mát của câu dễ ($p=0.9$) xuống 400 lần, dồn toàn bộ gradient bắt rung nơ-ron học cao khó ($p=0.6$).	$FL_1 = 0.000263$; $FL_2 = 0.020433$; FL dim câu dễ 400 lần; độ chênh lệch cao khó vọt lên 77.57 lần.
Câu 5	Cosine Annealing Schedule	Chiếc máy bay hạ cánh mượt mà; lượn sóng Cosine giảm learning rate trơn tru, giai đoạn đầu khám phá rộng, giai đoạn cuối trượt êm ái vào đáy chảo.	$t=0$: $lr=0.1000$; $t=50$: $lr=0.0505$; $t=100$: $lr=0.0010$; khắc phục nhược điểm giật cục của Step Decay.
Câu 6	Backpropagation Trong Mạng MLP	Dây chuyền sản xuất đập lỗi ngược; tính forward pass, đo sai số MSE, truyền tín hiệu lỗi ngược dòng tính trọn vẹn gradient 4 ma trận và cập nhật trọng số.	Forward $\hat{y} = 0.325$; Loss = 0.2278; Backward $\delta_2 = -0.675$; $\delta_1 = [-0.27, 0.2025]$; W_2 , b_2 , W_1 , b_1 mới.

BỨC TRANH TỔNG THỂ | Hệ Sinh Thái Tối Ưu Hóa Trong Deep Learning Hiện Đại

1. Hàm mất mát thông minh

Huber chống nhiễu ngoại lai; Focal Loss chống mất cân bằng lớp cực đoan (Object Detection).

2. Bộ tối ưu thích nghi

Momentum tích lũy quán tính; Adam tự thích nghi tốc độ học cho từng tham số riêng biệt.

3. Chiến lược học bền vững

Cosine Annealing hạ cánh mềm; Backpropagation tự động hóa dây chuyền sửa lỗi toàn mạng.

CÂU 1: CẬP NHẬT GRADIENT DESCENT VỚI MOMENTUM

Động Lực Học Quán Tính

ĐỀ BÀI GỐC Giả sử tại bước thời gian t , ta có:

Gradient của loss: $gt = [0.5, -1.2]$ • Vận tốc bước trước: $vt_{-1} = [0.1, -0.3]$
Hệ số momentum $\mu = 0.9$ • Tốc độ học $\eta = 0.01$ • Tham số $\theta_{t-1} = [1.0, -1.0]$

- a) Tính vận tốc mới vt sử dụng công thức: $vt = \mu * vt_{-1} + gt$
b) Cập nhật tham số θ_t sử dụng: $\theta_t = \theta_{t-1} - \eta * vt$
c) Giải thích ý nghĩa của việc sử dụng momentum trong việc ổn định quá trình cập nhật.

ÁN DỤ CHO BẾ HIỂU Quả Cầu Tuyết Lăn Xuống Đốc & Chiếc Xe Trượt Tuyết Có Quán Tính

Hãy tưởng tượng bạn trượt tuyết xuống một khe núi hẹp:

- Nếu không có quán tính (GD thuần túy):** Mỗi bước đi bạn chỉ nhìn theo cơn gió tạt ngang tức thời. Gió thổi sang trái bạn bước sang trái, gió tạt sang phải bạn bước sang phải. Kết quả là bạn bị **lắc lư zíc-zắc hỗn loạn giữa 2 vách núi**, say sóng và di chuyển cực kỳ chậm!
- Khi có quán tính (Momentum):** Bạn ngồi trên một cỗ xe trượt nặng trịch ($\mu = 0.9$). Chiếc xe giữ lại 90% vận tốc cũ. Hướng lao xuống dốc tích tụ ngày càng mạnh, còn các cơn gió tạt ngang đổi chiều liên tục sẽ **tự triệt tiêu lẫn nhau!** Hơn nữa, quán tính giúp xe phóng vọt qua những hố ga nhỏ (cực tiểu địa phương) mà không bị mắc kẹt!

CÂU 1A Tính Vận Tốc Mới vt Công thức: $vt = \mu * vt_{-1} + gt$ $vt = 0.9 * [0.1, -0.3] + [0.5, -1.2]$

- Chiều 1: $0.9(0.1) + 0.5 = 0.09 + 0.5 = \mathbf{0.59}$
- Chiều 2: $0.9(-0.3) + (-1.2) = -0.27 - 1.2 = \mathbf{-1.47}$

 $\Rightarrow$ Vận tốc mới: $vt = [0.59, -1.47]$ (Vận tốc cũ $[-0.3]$ cùng hướng âm với gradient mới $[-1.2]$ nên chiều thứ hai được gia tốc vọt lên $-1.47!$).**CÂU 1B** Cập Nhật Tham Số θ_t Công thức: $\theta_t = \theta_{t-1} - \eta * vt$ $\theta_t = [1.0, -1.0] - 0.01 * [0.59, -1.47]$

- $\theta_{t,1} = 1.0 - 0.0059 = \mathbf{0.9941}$
- $\theta_{t,2} = -1.0 - (-0.0147) = -1.0 + 0.0147 = \mathbf{-0.9853}$

 $\Rightarrow$ Tham số mới: $\theta_t = [0.9941, -0.9853]$

(Tham số dịch chuyển mượt mà ngược hướng dốc tổng hợp).

CÂU 1C • BÀN CHẤT SÂU SẮC Ba Ý Nghĩa Sống Còn Của Momentum Trong Tối Ưu Hóa Mạng Nơ-ron

- Triệt tiêu dao động zíc-zắc (Dampening Oscillations):** Trong các thung lũng hẹp (Ravines - dốc một bên rất đứng nhưng đáy thung lũng thoải dài), gradient bị giạt liên tục giữa hai sườn dốc. Vì các gradient này đổi dấu liên tục, tổng vận tốc vt trung bình theo chiều ngang bị triệt tiêu về gần 0, giúp mạng không bị văng khỏi khe núi.
- Gia tốc theo hướng nhất quán (Accelerating Convergence):** Dọc theo đáy khe dốc, các gradient liên tục chỉ về cùng một hướng. Hệ số $\mu = 0.9$ hoạt động như chuỗi cấp số nhân: $1 + 0.9 + 0.9^2 + \dots = 1/(1 - 0.9) = 10$ lần! Tốc độ hội tụ tăng vọt gấp 10 lần so với Gradient Descent truyền thống.
- Vượt qua điểm yên ngựa và hố cực tiểu nông (Escape Saddle Points & Local Minima):** Tại các lòng chảo phẳng lì (Saddle points), gradient triệt tiêu $gt \approx 0$ khiến GD đứng hình. Nhờ vận tốc tích lũy vt_{-1} từ trước, mô hình vẫn trượt thẳng qua vùng mù này và tiếp tục lăn xuống đáy tối ưu toàn cục.

CÂU 2: TÍNH TOÁN HÀM MẤT MẤT HUBER LOSS

Miễn Nhiệm Điểm Ngoại Lai

ĐỀ BÀI GỐC Cho sai số dự đoán trên 3 điểm dữ liệu: $e = [0.3, -1.5, 2.1]$. Sử dụng hàm mất mát Huber với $\delta = 1.0$:

$$L_{\delta}(e_i) = \frac{1}{2} \cdot e_i^2 \quad (\text{nếu } |e_i| \leq \delta) \quad \bullet \quad L_{\delta}(e_i) = \delta \cdot (|e_i| - \frac{1}{2} \cdot \delta) \quad (\text{nếu } |e_i| > \delta)$$

- a) Tính giá trị hàm mất mát Huber cho từng điểm dữ liệu.
b) Tính tổng loss trung bình (MSE) trên 3 điểm.
c) So sánh và giải thích sự khác biệt giữa loss của điểm thứ 3 ($e_3 = 2.1$) khi tính bằng Huber vs MSE. Tại sao Huber được gọi là "robust to outliers"?

ẢNH HƯỞNG CHO BẾ HIỂU Vị Quan Tòa Thông Thái & Tin Đồn Thất Thiệt

Hãy tưởng tượng một lớp học có một vài bạn báo cáo sai sót:

- Quan tòa MSE (Sai số bình phương):** Áp dụng luật "phạt theo bình phương". Ai sai một tí (0.3) thì phạt nhẹ (0.09). Nhưng nếu có một học sinh báo cáo con số khổng lồ vô lý ($e = 2.1$ - nghi ngờ do máy đo hỏng), MSE phạt nặng tới $2.1^2 = 4.41$! Mức phạt này áp đảo toàn bộ lớp, khiến cả lớp phải vất vả để bù đắp cho một tin đồn ngoại lai sai lệch!
- Quan tòa Huber (Thông thái & Tinh táo):** Nếu sai sót nhỏ trong ngưỡng bình thường ($|e| \leq 1.0$), quan tòa dùng bình phương êm ái. Nhưng nếu sai số vượt ngưỡng (> 1.0), quan tòa chuyển ngay sang mức phạt bậc nhất nhẹ nhàng (đường thẳng). Điểm ngoại lai không thể làm lật thuyền được!

CÂU 2A Tính Huber Loss Cho Từng Điểm ($\delta = 1.0$)

- Điểm $e_1 = 0.3$:** $|0.3| \leq 1.0 \Rightarrow$ Dùng nhánh bậc 2:
 $L_{\delta}(0.3) = \frac{1}{2} \times 0.3^2 = \frac{1}{2} \times 0.09 = \mathbf{0.0450}$
- Điểm $e_2 = -1.5$:** $|-1.5| = 1.5 > 1.0 \Rightarrow$ Dùng nhánh bậc 1:
 $L_{\delta}(-1.5) = 1.0 \times (1.5 - 0.5 \times 1.0) = 1.0 \times 1.0 = \mathbf{1.0000}$
- Điểm $e_3 = 2.1$:** $|2.1| = 2.1 > 1.0 \Rightarrow$ Dùng nhánh bậc 1:
 $L_{\delta}(2.1) = 1.0 \times (2.1 - 0.5 \times 1.0) = 1.0 \times 1.6 = \mathbf{1.6000}$

$\Rightarrow$ Vector Huber loss: $\mathbf{[0.045, 1.000, 1.600]}$

CÂU 2B Tính Tổng Loss Trung Bình MSE

Công thức chuẩn MSE = $(1/N) \sum e_i^2$:

- $e_1^2 = 0.3^2 = 0.09$
- $e_2^2 = (-1.5)^2 = 2.25$
- $e_3^2 = 2.1^2 = 4.41$

Tổng = $0.09 + 2.25 + 4.41 = 6.75$

MSE = $6.75 / 3 = \mathbf{2.2500}$

(Nếu dùng quy ước $1/(2N) \sum e_i^2$ thì MSE = 1.1250).

Nhận xét: Riêng điểm e_3 đóng góp tới $4.41 / 6.75 = \mathbf{65.3\%}$ tổng sai số của toàn bộ mô hình!

CÂU 2C • BÀN CHẤT SÂU SẮC So Sánh Điểm $e_3 = 2.1$ & Tại Sao Huber Loss Miễn Nhiệm Với Điểm Ngoại Lai (Robust to Outliers)?**1. Đối chiếu định lượng tại điểm cá biệt $e_3 = 2.1$:**

- MSE loss = $2.1^2 = \mathbf{4.4100}$ (vọt lên rất cao).
- Huber loss = $1.0 \times (2.1 - 0.5) = \mathbf{1.6000}$.

$\Rightarrow$ Mất mát MSE cao gấp **2.76 lần** so với Huber loss! MSE phóng đại cực hạn các sai số lớn do phép lũy thừa bậc 2.

2. Cơ chế kháng nhiễu (Robustness Mechanism):

- Đạo hàm theo sai số của MSE:** $d(\text{MSE})/de = 2e$. Với $e = 2.1$, lực kéo gradient là 4.2. Càng xa trung tâm, lực kéo càng tiến tới vô cùng, kéo vẹo toàn bộ đường hồi quy theo điểm nhiễu.

Đạo hàm của Huber: Khi $|e| > \delta$, $d(\text{Huber})/de = \delta \cdot \text{sign}(e) = \mathbf{\text{hằng số cố định } \pm 1.0}$! Dù dữ liệu có nhiễu tới $e = 1000$ thì gradient vẫn giữ trần cố định là 1.0, bảo vệ mô hình tuyệt đối không bị lật đổ.

CÂU 3: CẬP NHẬT ADAM OPTIMIZER

Chuẩn Mực Deep Learning Hiện Đại

ĐỀ BÀI GỐC Tại bước $t = 1$, cho:Gradient: $g_1 = [0.1, -0.4]$ • $\theta_0 = [0.0, 0.0]$ • $m_0 = [0, 0]$, $v_0 = [0, 0]$
Hyperparameters: $\eta = 0.001$ • $\beta_1 = 0.9$ • $\beta_2 = 0.999$ • $\epsilon = 10^{-8}$

- a) Tính moment bậc nhất m_1 và moment bậc hai v_1 đã hiệu chỉnh độ lệch (bias-corrected): $\hat{m}_1$, $\hat{v}_1$.
- b) Thực hiện cập nhật tham số θ_1 .
- c) Giải thích vai trò của ϵ trong công thức cập nhật Adam. Điều gì xảy ra nếu $\epsilon = 0$?

ÁNH DƯỠNG CHO BÉ HIỂU Vận Động Viên Leo Núi Thông Thái & Chiếc Phao Cứu Sinh Epsilon

Adam là sự kết hợp thiên tài giữa hai trợ lý:

- Trợ lý 1 (Moment bậc nhất m):** "Nhìn về hướng gió trung bình" (quán tính hướng đi).
- Trợ lý 2 (Moment bậc hai v):** "Đo độ gồ ghề của mặt đất" (tổng bình phương gradient). Nếu một con đường quá hiểm trở rung lắc dữ dội (v lớn), Adam sẽ **tự động chia cho căn bậc hai của v** để thu nhỏ bước chân, tránh trượt ngã. Nếu đường êm đềm (v nhỏ), Adam tự tin sải bước dài hơn!
- Chiếc phao ϵ :** Khi mặt đất bằng phẳng tuyệt đối không có độ dốc ($v = 0$), chiếc phao ϵ ở dưới mẫu số ngăn chặn thảm họa "chia cho số 0" khiến vận động viên ngất xỉu!

CÂU 3A Tính Moment & Hiệu Chính Độ Lệch Tại $t = 1$ 1. First Moment $\hat{m}_1$ (Quán tính hướng đi):

$$m_1 = \beta_1 * m_0 + (1 - \beta_1) * g_1 = 0.9(0) + 0.1 * [0.1, -0.4]$$

$$m_1 = [0.010, -0.040]$$

$$\text{Hiệu chỉnh: } \hat{m}_1 = m_1 / (1 - \beta_1^1)$$

$$\hat{m}_1 = [0.01, -0.04] / (1 - 0.9) = [0.10, -0.40]$$

2. Second Moment $\hat{v}_1$ (Độ rung bình phương):

$$v_1 = \beta_2 * v_0 + (1 - \beta_2) * g_1^2 = 0.999(0) + 0.001 * [0.1^2, (-0.4)^2]$$

$$v_1 = 0.001 * [0.01, 0.16] = [0.00001, 0.00016]$$

$$\text{Hiệu chỉnh: } \hat{v}_1 = v_1 / (1 - \beta_2^1)$$

$$\hat{v}_1 = v_1 / (1 - 0.999) = v_1 / 0.001 = [0.01, 0.16]$$

(Dấu mũ "hat" $\hat{m}$, $\hat{v}$ quy ước giá trị đã khử độ lệch Bias-Correction).CÂU 3B Cập Nhật Tham Số θ_1 (Chuẩn Hóa Bước Nhảy)Công thức: $\theta_1 = \theta_0 - \eta * \hat{m}_1 / (\sqrt{\hat{v}_1} + \epsilon)$ (Với $\theta_0 = [0, 0]$, $\eta = 0.001$, $\epsilon = 10^{-8} \approx 0$):

Tọa độ	Độ rung $\sqrt{\hat{v}_1}$	Giá trị mới θ_1
Chiều 1 ($\theta_{1,1}$)	$\sqrt{0.01} = 0.1$	$0 - 0.001 * (0.1/0.1) = -0.0010$
Chiều 2 ($\theta_{1,2}$)	$\sqrt{0.16} = 0.4$	$0 - 0.001 * (-0.4/0.4) = +0.0010$

 $\Rightarrow$ Vector mới: $\theta_1 = [-0.0010, +0.0010]$ Kỳ diệu: Dù $g_{1,2}$ (-0.4) mạnh gấp 4 lần $g_{1,1}$ (0.1), Adam tự chia cho độ rung $\sqrt{\hat{v}}$ để chuẩn hóa bước nhảy về đúng độ lớn $\eta = 0.001$!CÂU 3C • BẢN CHẤT SÂU SẮC Vai Trò Của Hằng Số $\epsilon = 10^{-8}$ Và Thảm Họa Khi $\epsilon = 0$

- Bảo vệ ổn định số học (Numerical Stability):** Trong công thức chia căn bậc hai $\hat{m} / (\sqrt{\hat{v}} + \epsilon)$, ϵ đóng vai trò là số hạng làm trơn (smoothing term). Nó đảm bảo mẫu số luôn dương nghiêm ngặt trong mọi tình huống tính toán của CPU/GPU.
- Điều gì xảy ra nếu $\epsilon = 0$?** Nếu $\epsilon = 0$, tại những tham số mà gradient bằng 0 liên tục (ví dụ các neuron bị khóa bởi ReLU hoặc các đặc trưng thừa hiếm xuất hiện), ta sẽ có $\hat{v} = 0$. Khi đó phép tính trở thành $\hat{m} / 0$ hoặc $0 / 0$!
- Hậu quả thực tế:** Máy tính sẽ lập tức sinh ra giá trị lỗi **NaN (Not a Number)** hoặc **Inf (Vô cùng)**. Giá trị rác này sẽ lan truyền ngược và lây nhiễm ra toàn bộ mạng nơ-ron qua các phép nhân ma trận tiếp theo, phá hủy hoàn toàn mô hình chỉ sau đúng 1 bước cập nhật!

CÂU 4: FOCAL LOSS CHO PHÂN LOẠI NHỊ PHÂN

Đột Phá Thị Giác Máy Tính (RetinaNet)

ĐỀ BÀI GỐC Xét bài toán binary classification. Cho 2 điểm dữ liệu cùng nhãn thực $y = 1$:

- Điểm 1: $y_1 = 1$, xác suất $p_1 = 0.9$ (Dễ phân loại đúng)
 - Điểm 2: $y_2 = 1$, xác suất $p_2 = 0.6$ (Khó phân loại hơn)
- Tham số Focal Loss: $\gamma = 2.0$, $\alpha = 0.25$ • Công thức: $FL(pt) = -\alpha * (1 - pt)^\gamma * \log(pt)$

- Tính Focal Loss cho từng điểm dữ liệu.
- So sánh Focal Loss với Cross-Entropy loss tiêu chuẩn (tính CE loss cho cả hai điểm).
- Dựa trên kết quả, giải thích cơ chế "hard example mining" của Focal Loss.

ẤN DỤ CHO BẾ HIỂU Người Thầy Ôn Thi Thông Minh & 1000 Câu Hỏi Trắc Nghiệm Dễ

Hãy tưởng tượng bạn đang ôn thi:

- Trong đề thi có **999 câu cực dễ** ($1 + 1 = 2$, robot đoán tự tin 90%) và chỉ có **1 câu cực khó** (robot đoán 60%).
- Nếu dùng Cross-Entropy thường:** Dù mỗi câu dễ chỉ bị trừ điểm rất nhỏ, nhưng lấy số nhỏ đó nhân với 999 câu dễ thì tổng điểm trừ của câu dễ vẫn đè bẹp câu khó! Robot sẽ dành trọn ngày chỉ để ôn lại câu dễ mà bỏ quên hoàn toàn câu khó!
- Focal Loss xuất hiện:** Dùng hệ số giảm âm $(1 - p)^\gamma$. Câu dễ ($p=0.9$) bị giảm điểm phạt xuống **400 lần** (gần như biến mất). Toàn bộ tâm trí của người thầy được dồn vào **1 câu hỏi khó duy nhất!**

CÂU 4A & 4B Tính & So Sánh Định Lượng FL vs CE(Quy chuẩn AI sử dụng log tự nhiên $\ln$, $\alpha = 0.25$, $\gamma = 2$):**1. Điểm 1: Dễ phân loại ($p_1 = 0.9$):**

- CE1 = $-\ln(0.9) \approx 0.1054$
- Hệ số điều chế: $(1 - 0.9)^2 = 0.1^2 = 0.01$

$$FL1 = 0.25 \times 0.01 \times 0.1054 = 0.000263$$

 $\Rightarrow$ FL1 làm suy giảm mất mát CE tới: $0.1054 / 0.000263 \approx 400$ lần!**2. Điểm 2: Khó phân loại ($p_2 = 0.6$):**

- CE2 = $-\ln(0.6) \approx 0.5108$
- Hệ số điều chế: $(1 - 0.6)^2 = 0.4^2 = 0.16$

$$FL2 = 0.25 \times 0.16 \times 0.5108 = 0.020433$$

 $\Rightarrow$ FL2 chỉ làm giảm mất mát CE: $0.5108 / 0.020433 \approx 25$ lần.**ĐỐI CHIẾU BẢNG SỐ LIỆU** Sự Thay Đổi Cục Diện Giữa Hai Điểm

Chỉ Số	Điểm 1 (Dễ, $p=0.9$)	Điểm 2 (Khó, $p=0.6$)	Tỷ Lệ (Khó / Dễ)
Cross-Entropy	0.1054	0.5108	4.85 lần
Hệ số $(1 - pt)^2$	0.0100 (phạt siêu nhẹ)	0.1600 (giữ lại đáng kể)	16.00 lần
Focal Loss	0.000263	0.020433	77.57 lần!

Phát hiện chấn động: Trong Cross-Entropy, mẫu khó chỉ quan trọng hơn mẫu dễ 4.85 lần. Nhưng qua lăng kính Focal Loss, mẫu khó đã quan trọng hơn gấp **77.57 lần!** Mẫu dễ hoàn toàn bị vô hiệu hóa!

CÂU 4C • BẢN CHẤT SÂU SẮC Cơ Chế "Hard Example Mining" Tự Động Hóa Tuyệt Vời Của Focal Loss

- Vấn đề mất cân bằng tiền cảnh / hậu cảnh (Foreground-Background Imbalance):** Trong nhận diện vật thể (Object Detection), mỗi bức ảnh có tới 100,000 ô neo (anchors) là nền trời đất trống (Easy Negatives, $p \geq 0.99$) và chỉ có vài ô chứa đồ vật thật. Hàng trăm nghìn mẫu dễ cộng dồn gradient lại sẽ nhấn chìm toàn bộ tín hiệu học tập của vật thể quý giá.
- Bộ lọc động tự nhiên (Dynamic Modulating Factor):** Hệ số $(1 - pt)^\gamma$ hoạt động như một bộ giảm thanh tự động. Khi $pt \rightarrow 1$, hệ số này tiến về 0 theo hàm lũy thừa bậc γ , dập tắt gradient của những mẫu đã học xong. Khi $pt \leq 0.5$, hệ số này giữ giá trị lớn, truyền nguyên vẹn xung lực gradient để chỉnh sửa mạng.
- Không cần thuật toán lấy mẫu phức tạp:** Trước đây người ta phải dùng kỹ thuật OHEM (Online Hard Example Mining) thủ công rất tốn RAM và tính toán. Focal Loss giải quyết trọn vẹn bài toán hóc búa này chỉ bằng một dòng code hàm mất mát duy nhất!

CÂU 5: COSINE ANNEALING LEARNING RATE SCHEDULE

Điều Phối Tốc Độ Học Tối Ưu

ĐỀ BÀI GỐC Xét lịch trình học Cosine Annealing không có Warm Restarts:

$$\eta_{\max} = 0.1 \cdot \eta_{\min} = 0.001 \cdot \text{Tổng số epoch: } T_{\max} = 100$$
$$\text{Công thức: } \eta_t = \eta_{\min} + \frac{1}{2} * (\eta_{\max} - \eta_{\min}) * (1 + \cos(\pi * t / T_{\max}))$$

- a) Tính learning rate tại epoch $t = 0, t = 50, t = 100$.
b) Vẽ phác thảo đồ thị learning rate theo epoch (mô tả định tính).
c) Giải thích lợi ích của việc sử dụng Cosine Annealing so với Step Decay truyền thống.

ẢN DỤ CHO BẾ HIỂU Chiếc Máy Bay Hạ Cánh Xuống Đường Bằng vs Nhảy Cầu Thang Giật Cục

Hãy tưởng tượng bạn đang điều khiển máy bay hạ cánh:

- Kiểu cũ (Step Decay):** Máy bay đang bay nhanh, bạn giật phanh đột ngột làm nó rớt xuống 1 tầng; bay tiếp rồi lại giật phanh rớt xuống tầng nữa. Những cú rơi bậc thang này gây sốc áp suất, hành khách nôn nao và máy bay rất dễ đâm sầm xuống đất!
- Kiểu mới (Cosine Annealing):** Máy bay lướt theo một đường cong Cosine mềm mại như dải lụa. Lúc đầu ở trên cao lướt êm, đoạn giữa hạ độ cao thoải mái, và đoạn cuối là đà tiếp đất êm ái không hề có một rung chấn nhỏ nào!

CÂU 5A Tính Toán Tốc Độ Học Tại Các EpochHiệu số: $\eta_{\max} - \eta_{\min} = 0.1 - 0.001 = 0.099$.Biên độ: $1/2 \times 0.099 = 0.0495$.**• Tại epoch $t = 0$ (Bắt đầu):**

$$\cos(0) = 1 \Rightarrow 1 + \cos(0) = 2$$

$$\eta_0 = 0.001 + 0.0495 \times 2 = 0.001 + 0.099 = 0.1000 \quad (\eta_{\max})$$

• Tại epoch $t = 50$ (Chính giữa quá trình):

$$\cos(50\pi/100) = \cos(\pi/2) = 0 \Rightarrow 1 + \cos(\pi/2) = 1$$

$$\eta_{50} = 0.001 + 0.0495 \times 1 = 0.001 + 0.0495 = 0.0505$$

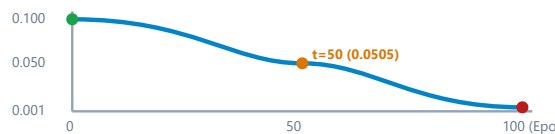
• Tại epoch $t = 100$ (Kết thúc):

$$\cos(100\pi/100) = \cos(\pi) = -1 \Rightarrow 1 + \cos(\pi) = 0$$

$$\eta_{100} = 0.001 + 0.0495 \times 0 = 0.001 + 0 = 0.0010 \quad (\eta_{\min})$$

CÂU 5B Mô Tả Định Tính Đồ Thị η_t Theo Epoch

Đồ thị là một nửa chu kỳ sóng Cosine trơn (S-curve lật ngược):



- Giai đoạn đầu (0 - 20):** Giảm rất chậm, giữ tốc độ lớn để tự do khám phá không gian tham số rộng lớn.
- Giai đoạn giữa (20 - 80):** Đốc xuống nhanh để nhanh chóng tiếp cận thung lũng nghiệm tối ưu.
- Giai đoạn cuối (80 - 100):** Mặt đáy thoải phẳng, giảm tốc độ cực nhỏ để rà soát tinh chỉnh vi mô quanh đáy vực.

CÂU 5C • BẢN CHẤT SÂU SẮC Ba Lợi Ích Vượt Trội Của Cosine Annealing So Với Step Decay Truyền Thống

- Loại bỏ cú sốc gradient (No Destructive Shocks):** Trong Step Decay, việc chia tốc độ học cho 10 tại các epoch cố định (như epoch 30, 60) tạo ra sự đứt gãy đột ngột. Mô hình có thể bị kẹt lại ngay tại vùng lồi xấu mà nó vừa rơi vào. Cosine Annealing biến đổi liên tục trơn tru, giúp mô hình thích nghi nhẹ nhàng.
- Không cần phỏng đoán siêu tham số (Hyperparameter-Free Milestones):** Với Step Decay, kỹ sư phải vô đầu bứt tai chọn xem nên giảm ở epoch nào (30? 40? hay 75?). Với Cosine Annealing, lịch trình tự động dần trải hoàn hảo từ epoch 0 đến epoch cuối cùng dựa trên hàm lượng giác tự nhiên.
- Hội tụ sâu vào đáy cực tiểu phẳng (Flat Minima Convergence):** Đạo hàm của Cosine bằng 0 tại cả 2 đầu mút $t = 0$ và $t = T_{\max}$. Ở những epoch cuối, learning rate giảm chậm dần và duy trì đều đặn quanh $\eta_{\min}$, giúp trọng số hội tụ chính xác vào tâm của các vùng cực tiểu phẳng (Flat Minima) - nơi mô hình đạt khả năng tổng quát hóa (Generalization) cao nhất trên tập dữ liệu thực tế!

CÂU 6: BACKPROPAGATION TRONG MLP ĐƠN GIẢN

Trái Tim Của Deep Learning

ĐỀ BÀI GỐC Xét mạng MLP 1 lớp ẩn với các tham số:

$$W(1) = \begin{bmatrix} 0.5 & -0.2 \\ 0.3 & 0.1 \end{bmatrix}, \quad b(1) = \begin{bmatrix} 0.1 & -0.1 \end{bmatrix}^T$$
$$W(2) = \begin{bmatrix} 0.4 & -0.3 \end{bmatrix}, \quad b(2) = 0.2$$

• Hàm kích hoạt ReLU • Đầu vào $x = \begin{bmatrix} 1.0 & 0.5 \end{bmatrix}^T$
Nhãn thực: $y = 1$ • Hàm mất mát MSE Loss: $L = 1/2 * (\hat{y} - y)^2$ • Learning rate $\eta = 0.1$

- a) Thực hiện forward pass để tính đầu ra $\hat{y}$.
b) Tính loss L giữa $\hat{y}$ và y .
c) Thực hiện backward pass để tính gradient cho $W(2)$, $b(2)$, $W(1)$, $b(1)$.
d) Cập nhật các tham số với learning rate $\eta = 0.1$.

CÂU 6A & 6B Forward Pass & Tính Mất Mát L

1. Lớp ẩn 1: $z(1) = W(1)x + b(1)$
 $z1(1) = 0.5(1.0) - 0.2(0.5) + 0.1 = 0.5 - 0.1 + 0.1 = 0.50$
 $z2(1) = 0.3(1.0) + 0.1(0.5) - 0.1 = 0.3 + 0.05 - 0.1 = 0.25$
 $a(1) = \text{ReLU}(z(1)) = \max(0, z(1)) = \begin{bmatrix} 0.50 & 0.25 \end{bmatrix}^T$
2. Lớp ra: $z(2) = W(2)a(1) + b(2)$
 $\hat{y} = 0.4(0.50) - 0.3(0.25) + 0.2 = 0.20 - 0.075 + 0.20$
 $\hat{y} = \begin{bmatrix} 0.3250 \end{bmatrix}$
3. MSE Loss L : $L = 1/2 * (\hat{y} - y)^2$
 $L = 1/2 * (0.325 - 1.0)^2 = 1/2 * (-0.675)^2 = 1/2 * 0.455625$
 $L = \begin{bmatrix} 0.2278 \end{bmatrix}$ (Nếu không có $1/2$ thì $L = 0.4556$).

CÂU 6C Backward Pass Tính Gradient Từng Lớp

1. Sai số lớp ra $\delta(2)$:
 $\delta(2) = \partial L / \partial z(2) = \hat{y} - y = 0.325 - 1.0 = \begin{bmatrix} -0.6750 \end{bmatrix}$
 $\partial L / \partial W(2) = \delta(2) * (a(1))^T = -0.675 * \begin{bmatrix} 0.50 & 0.25 \end{bmatrix}$
 $\partial L / \partial W(2) = \begin{bmatrix} -0.3375 & -0.16875 \end{bmatrix}$
 $\partial L / \partial b(2) = \delta(2) = \begin{bmatrix} -0.6750 \end{bmatrix}$
2. Sai số lan truyền ngược về lớp ẩn $\delta(1)$:
Vì $z(1) = \begin{bmatrix} 0.5 & 0.25 \end{bmatrix} > 0$ nên $\text{ReLU}'(z(1)) = \begin{bmatrix} 1 & 1 \end{bmatrix}^T$.
 $\delta(1) = \begin{bmatrix} W(2) \end{bmatrix}^T * \delta(2) \odot \text{ReLU}'(z(1))$
 $\delta(1) = \begin{bmatrix} 0.4 * (-0.675) & -0.3 * (-0.675) \end{bmatrix}^T = \begin{bmatrix} -0.2700 & +0.2025 \end{bmatrix}^T$
 $\partial L / \partial W(1) = \delta(1) * x^T = \begin{bmatrix} -0.2700 & 0.2025 \end{bmatrix}^T * \begin{bmatrix} 1.0 & 0.5 \end{bmatrix}$
 $\partial L / \partial W(1) = \begin{bmatrix} -0.2700 & -0.1350 \end{bmatrix}, \begin{bmatrix} +0.2025 & +0.10125 \end{bmatrix}$
 $\partial L / \partial b(1) = \delta(1) = \begin{bmatrix} -0.2700 & +0.2025 \end{bmatrix}^T$

CÂU 6D Cập Nhật Tham Số Toàn Mạng Với Learning Rate $\eta = 0.1$ ($\theta_{\text{mới}} = \theta_{\text{cũ}} - \eta * \text{grad}(L)$)

1. Cập nhật tham số Lớp Ra:
- $W(2)_{\text{mới}} = \begin{bmatrix} 0.4 & -0.3 \end{bmatrix} - 0.1 * \begin{bmatrix} -0.3375 & -0.16875 \end{bmatrix}$
 $W(2)_{\text{mới}} = \begin{bmatrix} 0.4 + 0.03375 & -0.3 + 0.016875 \end{bmatrix} = \begin{bmatrix} 0.43375 & -0.283125 \end{bmatrix}$
 - $b(2)_{\text{mới}} = 0.2 - 0.1 * (-0.675) = 0.2 + 0.0675 = \begin{bmatrix} 0.2675 \end{bmatrix}$
2. Cập nhật tham số Lớp Ẩn:
- $W(1)_{\text{mới}} = \begin{bmatrix} 0.5 & -0.2 \\ 0.3 & 0.1 \end{bmatrix} - 0.1 * \begin{bmatrix} -0.2700 & -0.1350 \\ 0.2025 & 0.10125 \end{bmatrix}$
 $W(1)_{\text{mới}} = \begin{bmatrix} 0.5270 & -0.1865 \\ 0.27975 & 0.089875 \end{bmatrix}$
 - $b(1)_{\text{mới}} = \begin{bmatrix} 0.1 & -0.1 \end{bmatrix}^T - 0.1 * \begin{bmatrix} -0.2700 & 0.2025 \end{bmatrix}^T = \begin{bmatrix} 0.1270 & -0.12025 \end{bmatrix}^T$

ĐÚC KẾT SỰ PHẠM TOÀN DIỆN Bản Chất Chuỗi Phản Ứng Dây Chuyền Backpropagation

Toàn bộ quá trình Backpropagation chính là một chiếc thước đo trách nhiệm: Sai số ở đầu ra ($\delta_2 = -0.675$) được phân phối ngược về lớp ẩn thông qua các "đường ống dẫn" chính là trọng số $W(2)$. Trọng số nào kết nối mạnh hơn sẽ phải chịu trách nhiệm sửa sai nhiều hơn. Nhờ quy tắc chuỗi vi tích phân, hàng tỷ tham số trong các mô hình AI nghìn tỷ tham số như GPT hay Llama đều được cập nhật song song cực nhanh chỉ trong vài mili-giây!